



FedBook-Sem

Technical Walkthrough — Architecture, Stack, Data Flow

For engineers and technical reviewers

What FedBook-Sem Is

- FedBook-Sem is a federated social layer for an intelligent online bookstore. Users own an ActivityPub actor, write W3C Web Annotations on book passages, and receive recommendations that fuse semantic similarity, collaborative filtering, and privacy-preserving cross-platform reception.
- Two independent runtimes (Node backend, Python ML service) share one Neo4j.
- Streamlit dashboard is a thin client over the ML FastAPI.
- Idempotent seeders — FE seed and ML seed can co-exist safely.

Runtime Topology

localhost:3000	React FE (CRA dev server, proxies /api → :3001)
-----------------------	---

localhost:3001	Express backend (Node 20)
-----------------------	---------------------------

localhost:7474 / 7687	Neo4j 5.15 (browser + bolt)
------------------------------	-----------------------------

localhost:6379	Redis 7
-----------------------	---------

localhost:5432	PostgreSQL 16
-----------------------	---------------

localhost:8000	FastAPI ML service (Python 3.11)
-----------------------	----------------------------------

localhost:8501	Streamlit dashboard
-----------------------	---------------------

Technology Stack

Federation	ActivityPub + WebFinger + HTTP Signatures (RSA-2048)
Annotations	W3C Web Annotation Data Model — TextQuote + TextPosition selectors
Backend	Node.js 20 · Express · JWT · bcrypt · node-forge · multer · Bull
Frontend	React 18 · React Router v6 · Axios · custom dark design system
Graph DB	Neo4j 5.15 + APOC + Graph Data Science + native vector index
ML service	Python 3.11 · FastAPI · sentence-transformers · LightFM · LightGBM
Sentiment	CardiffNLP twitter-roberta-base-sentiment-latest (RoBERTa)
Cross-encoder	BAAI/bge-reranker-base (~500 MB)
Dashboard	Streamlit — Recommender demo + Model registry
Infrastructure	Docker Compose · Redis 7 · PostgreSQL 16

System Diagram

Open docs/presentation/FedBookSem_Architecture.drawio in draw.io or app.diagrams.net for the layered architecture with all services, endpoints, and data flows.

— *Client · Application · Ingestion · Data · External layers*

Layered Architecture — Five Tiers

Client	React FE (:3000) · Streamlit dashboard (:8501) · remote ActivityPub servers
Application	Express backend (:3001) · FastAPI ML service (:8000) · ActivityPub inbox / outbox / WebFinger
Ingestion	Reddit / YouTube / Bluesky / Mastodon collectors → entity resolver → sentiment scorer → aggregator
Data	Neo4j 5.15 (:7687) · Redis 7 (:6379) · PostgreSQL 16 (:5432) · cached model files on disk
External	Open Library · Hardcover GraphQL · HuggingFace Hub · Kaggle 7k · goodbooks-10k · remote AP peers

Request Lifecycle — Login and Feed

- Browser POSTs /api/auth/login → CRA dev proxy (setupProxy.js) forwards to Express :3001.
- Express: MATCH (:Person {username}), bcrypt.compare(password, passwordHash), jwt.sign(actor, JWT_SECRET, '7d').
- JWT stored in localStorage; axios interceptor at src/api/client.js attaches Bearer to every subsequent call.
- GET /api/feed → verifyJwt middleware → single Cypher: MATCH (me)-[:FOLLOWS]->(f)-[:AUTHORED]->(r)-[:REVIEWS]->(b) RETURN ...
- Response JSON hydrated client-side, rendered by Feed page component.

Federation Flow — Outbound Follow

- POST /api/social/follow {targetId: 'acct:bob@remote.social'}.
- backend/src/routes/social.js → WebFinger lookup: GET https://remote.social/.well-known/webfinger?resource=acct:bob@remote.social.
- Fetch actor JSON from returned URL; extract publicKey and inbox.
- activitypub/delivery.js builds Follow activity, canonicalises headers, signs with alice.privateKey (RSA-SHA256).
- Signature header: keyId=, algorithm=rsa-sha256, headers=(request-target) host date digest, signature=.
- POST to bob's inbox with Content-Type application/activity+json; expect 202 Accepted.

Federation Flow — Inbound Activity

- Remote server POSTs Follow / Like / Announce / Create to /inbox.
- backend/src/routes/inbox.js: parse Signature header, fetch remote actor's publicKey via keyId URL.
- Verify canonicalised headers against signature; reject on mismatch.
- Dispatch by activity type: Follow → MERGE (:FOLLOWS) + send Accept; Like → MERGE (:LIKES); Create Review → MERGE (:Review) + REVIEWS edge.
- All writes go through the same Neo4j driver used by the FE routes — one graph, many writers.

Recommender Request Flow — /recommend/similar

- GET /recommend/similar?isbn=X&rerank;=ltr&limit;=10.
- VectorSearcher: read :Book(embedding) for X (or encode ?text= with all-MiniLM-L6-v2).
- Neo4j: CALL db.index.vector.queryNodes('bookEmbedding', 50, \$emb) YIELD node, score.
- ReceptionScorer: MATCH (b)-[:RECEPTION_ON]->(r) for candidate ISBNs → per-platform sentiment (Reddit .35, YT .25, BS/Ma .20).
- Cross-encoder (bge-reranker-base) computes ce_score for query × candidate pairs.
- LightGBM LambdaRank (ltr.pkl) fuses (sim, cf, reception, diversity, ce, freshness) in fixed FEATURE_ORDER.
- Sort by fused score, take top-N, optional Open Library / Hardcover enrichment layer.

Ingestion Data Flow (privacy-preserving)

- `ingest_daily.py --platforms ... --limit N` — cron or manual.
- `MATCH (b:Book) WHERE b.description IS NOT NULL AND b.thumbnail IS NOT NULL LIMIT $limit`.
- For each (book × collector): call platform API (PRAW / YouTube DataAPI / AT Protocol / Mastodon timeline).
- `SentimentScorer.batch (twitter-roberta, batch=16)` → labels aggregated via `scorer.aggregate`.
- Persist: `MERGE (b)-[:RECEPTION_ON]->(r:PlatformReception {platform, book_isbn}) SET positive, neutral, negative, mentions, external_ids, ingested_at, expires_at (YouTube +30d), demo=false`.
- Raw mention text never written; `external_ids` kept only for deletion sweeps.

Neo4j Data Model — Nodes

:Person	id, username, displayName, bio, domain, publicKey, privateKey, passwordHash, avatarUrl, createdAt
:Book	id, isbn, title, author, year, description, thumbnail, subjects, embedding (vector 384-d), goodbooksBookId, sourceCatalog
:Review	id, content, rating (1-5), published, activityId, seedKey?
:Annotation	id, motivation, bodyValue, exactText, prefix, suffix, startOffset, endOffset, bookSource, created, seedKey?
:PlatformReception	platform, book_isbn, positive, neutral, negative, mentions, external_ids, ingested_at, expires_at, demo

Neo4j Data Model — Relationships & Indexes

Social	<code>(:Person)-[:FOLLOWS]->(:Person) · (:Person)-[:LIKES]->(:Review)</code>
Authoring	<code>(:Person)-[:AUTHORED]->(:Review) · (:Person)-[:ANNOTATED]->(:Annotation)</code>
Content targets	<code>(:Review)-[:REVIEWS]->(:Book) · (:Annotation)-[:ON_SOURCE]->(:Book)</code>
Reception	<code>(:Book)-[:RECEPTION_ON]->(:PlatformReception)</code>
Constraints	<code>UNIQUE :Person(username) · UNIQUE :Book(isbn)</code>
Vector index	<code>bookEmbedding on :Book(embedding), cosine, 384 dims (MiniLM-L6-v2)</code>
GDS	<code>projection :Person → :Person for personalised PageRank in /recommend/graph</code>

Deployment Topology

React FE	native · react-scripts start · :3000 · CRA proxy → :3001
Express BE	native · nodemon src/app.js · :3001
Neo4j 5.15	docker · fedbooksem-neo4j · :7474 + :7687 · APOC + GDS · volumes: neo4j_data, neo4j_logs
Redis 7	docker · fedbooksem-redis · :6379
PostgreSQL 16	docker · fedbooksem-postgres · :5432 · volume: postgres_data
ML service	docker (built) · fedbook-ml · :8000 · volumes: ml_hf_cache, ml_torch_cache, ./ml-service
Dashboard	docker (built) · fedbook-ml-dashboard · :8501 · streamlit entrypoint

Port Map (Host → Container / Process)

3000	React CRA dev server (native)
3001	Express backend (native)
7474	Neo4j HTTP browser UI (docker)
7687	Neo4j bolt (docker)
6379	Redis (docker)
5432	PostgreSQL (docker)
8000	FastAPI ML service (docker)
8501	Streamlit dashboard (docker)

Model Registry (loaded at ML startup)

all-MiniLM-L6-v2	sentence-transformers · semantic embeddings (384-d) · ~90 MB · ml_hf_cache
bge-reranker-base	BAAI cross-encoder · query × candidate re-scoring · ~500 MB · ml_hf_cache
twitter-roberta-base-sentiment-latest	CardiffNLP RoBERTa · sentiment for ingestion · ~500 MB · ml_hf_cache
lightfm.pkl	LightFM WARP · collaborative filtering · ~10 MB · ml-service/models/ · trained off-machine
ltr.pkl	LightGBM LambdaRank · learned rank fusion · ~1 MB · ml-service/models/

Environment Configuration

DOMAIN, BASE_URL	Federation identity — used as ActivityPub actor IDs (e.g. <code>http://localhost:3001</code>)
NEO4J_URI / USER / PASSWORD	Shared by backend and ML — <code>bolt://localhost:7687</code> native, <code>bolt://neo4j:7687</code> in-docker
JWT_SECRET	Backend HS256 signing key — required or <code>/api/auth/login</code> crashes
ML_SERVICE_URL YOUTUBE_API_KEY /	Backend → ML gateway (<code>http://ml-service:8000</code> in docker network)
REDDIT_CLIENT_ID+SECRET / BSKY_HANDLE+APP_PASSWORD	Ingestion auth (Bluesky and Mastodon work anonymously)
HARDCOVER_API_TOKEN	Optional — live star ratings in book cards
MASTODON_INSTANCES	Comma-separated hashtag-timeline hosts (default: <code>mastodon.social</code> , <code>ohai.social</code>)
HF_HUB_OFFLINE, TRANSFORMERS_OFFLINE	Force cached HuggingFace weights — no network at startup

Security & Signature Design

- Signing keys — RSA-2048 pair generated at user creation by node-forge, stored on the :Person node (publicKey + privateKey).
- Outbound activity — HTTP Signatures over (request-target) host date digest; algorithm=rsa-sha256; keyId links to publicKey URL.
- Inbound activity — /inbox fetches remote actor's publicKey via keyId, verifies canonicalised headers, rejects on mismatch.
- JWT — HS256 with JWT_SECRET, 7-day expiry; payload = {id, username, displayName, bio, domain, avatarUrl}.
- Passwords — bcryptjs, cost factor 12; comparison in constant time.
- Helmet middleware sets CSP, HSTS, X-Frame-Options, etc.; response headers confirm.

Data Contracts (JSON-LD / JSON shapes)

Actor (AP)	<code>{@context, type: "Person", id, preferredUsername, inbox, outbox, followers, following, publicKey: {id, owner, publicKeyPem}}</code>
Follow (AP)	<code>{@context, type: "Follow", actor, object: , id}</code>
Web Annotation	<code>{@context, type: "Annotation", motivation, body: {type: "TextualBody", value}, target: {source, selector: [TextQuoteSelector, TextPositionSelector]}}</code>
PlatformReception	<code>{platform, book_isbn, positive, neutral, negative, mentions, external_ids, ingested_at, expires_at, demo}</code>
JWT payload	<code>{id, username, displayName, bio, domain, avatarUrl, iat, exp}</code>

Express Backend — Route Groups

- `/api/auth` — POST `/login` → JWT (7d) signed with `process.env.JWT_SECRET`.
- `/api/reviews` — CRUD + like + announce + replies; federates via ActivityPub CREATE.
- `/api/annotations` — W3C Web Annotation CRUD with TextQuote + TextPosition selectors.
- `/api/feed` — personalised, all, recommendations, liked-by-followed.
- `/api/books`, `/api/users`, `/api/social`, `/api/covers` (SVG cover generator).
- `/users/:u`, `/inbox`, `/.well-known/webfinger` — ActivityPub actor + WebFinger surface.

React Frontend — Composition

- Create React App with a custom setupProxy.js forwarding /api, /avatars, /users, /books, /.well-known, /inbox → :3001.
- Single axios client (src/api/client.js) with a JWT bearer interceptor.
- React Router v6 with pages: Feed, Book, BookReader, Profile, Login, Register, Followers, Following, User.
- Custom design system — Playfair Display + DM Sans over CSS variables.

ML Service — FastAPI Endpoints

GET /health	Model load state (transformer, cross-encoder, LightFM, LTR)
GET /books/search	Fuzzy title / author lookup
GET /books/details	Catalogue + reception + optional live enrichment
GET /recommend/similar	Semantic k-NN with optional cross-encoder / linear / LTR re-rank
GET /recommend/cf	LightFM WARP — 503 if lightfm.pkl missing
GET /recommend/graph	Neo4j GDS k-NN and personalised PageRank

Recommender Strategies (all live)

Semantic	MiniLM-L6-v2 embeddings + Neo4j native vector index (cosine k-NN)
Cross-encoder rerank	BAAI/bge-reranker-base for query × candidate re-scoring
Collaborative filtering	LightFM WARP trained on goodbooks-10k (~6M real ratings)
Graph traversal	Neo4j GDS k-NN and personalised PageRank
Cross-platform reception	Weighted sentiment across Reddit / YouTube / Bluesky / Mastodon
Learned rank fusion	LightGBM LambdaRank over 6 features (sim, cf, reception, diversity, ce, freshness)

Sentiment Pipeline

- CardiffNLP twitter-roberta-base-sentiment-latest — batch=16, single or batch inference.
- Twitter-style preprocessing: @username → @user, URLs → http.
- Offline fallback lexicon scorer activates when HF hub or model file is unavailable — labels neutral by default, requires strong signal to shift.
- Aggregation returns (positive, neutral, negative) counts per (book, platform).

Entity Resolution

- rapidfuzz WRatio fuzzy match on title + author, default min_score = 82.
- In-memory catalogue index built at startup, refreshed periodically.
- Maps user-generated mention text (misspelled, no ISBN) to canonical :Book.isbn before scoring.

Cross-Platform Ingestion

Reddit	PRAW OAuth · subreddits /r/books, /r/booksuggestions, /r/52book, /r/literature
YouTube	Data API v3 · top-level comments on review videos · 30-day retention
Bluesky	AT Protocol public searchPosts · optional app-password auth
Mastodon	Public hashtag timelines · #bookstodon, #booksky, #booktok, #bookreview

Storage Model — Neo4j Labels & Relationships

- Nodes: :Person, :Book, :Review, :Annotation, :PlatformReception.
- Relationships: :FOLLOWS, :AUTHORED, :REVIEWS, :LIKES, :ANNOTATED, :ON_SOURCE, :RECEPTION_ON.
- Native vector index on :Book(embedding) for semantic k-NN.
- GDS graph projection over :Person → :Person for personalised PageRank.

Federation Details

- ActivityPub actor JSON-LD at /users/:username with publicKey block for HTTP Signatures.
- WebFinger at /.well-known/webfinger resolves acct:user@domain to actor URL.
- Outbound activities signed with node-forge (RSA-2048, private key stored on :Person node).
- Shared inbox at /inbox verifies signature, dispatches Follow / Like / Announce / Create.

Deployment — Docker Compose

- Services: neo4j, redis, postgres, ml-service, ml-dashboard.
- ml-service and ml-dashboard both build from ml-service/Dockerfile; dashboard runs streamlit.
- Volumes: neo4j_data, postgres_data, ml_hf_cache, ml_torch_cache — model + DB state persists across rebuilds.
- Backend + frontend run natively via nodemon / react-scripts for hot reload.

Operations

- `npm run seed` — idempotent FE seed (3 users, 6 Sri Lankan books, follows, reviews, annotations).
- `seed_kaggle_books.py` — Kaggle 7k catalogue via MERGE (no wipe).
- `build_embeddings.py` — MiniLM embeddings written to `:Book(embedding)`.
- `ingest_daily.py --platforms ... --limit N` — real cross-platform ingestion, `--drop-mock` replaces demo data.
- `purge_expired_receptions.py` — daily YouTube 30-day retention purge.
- `reddit_deletion_sweep.py` — weekly, honours Reddit deletions.

Privacy & Compliance Design

- Raw mention text is never persisted — only aggregated counts and external IDs for deletion sweep.
- YouTube receptions carry `expires_at`, purged after 30 days.
- Reddit deletion sweep removes counts for posts that no longer exist.
- All external calls carry a project user-agent identifying the researcher.

Where to Look in the Repo

backend/src/routes/*.js · backend/src/activitypub/delivery.js
ml-service/src/fedbook_ml/api.py · ml-service/src/fedbook_ml/ingest/*.py
ml-service/src/fedbook_ml/dashboard/pages/*.py

— *docs/SETUP_ML_AND_DASHBOARD.md* walks the whole bring-up.